

problem 1:- Read numbers from input.txt, find sum upto each number, write to output.txt.

code:

```
import java.io.*;  
import java.util.*;
```

```
public class sum {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(new File("input.txt"));
```

```
        PrintWriter pw = new PrintWriter(new File("output.txt"));
```

```
        sc.useDelimiter(",");
```

```
        int n = sc.nextInt();
```

```
        int sum = n * (n + 1) / 2;
```

```
        pw.print(sum);
```

```
        if (sc.hasNext()) {
```

```
            pw.print(", ");
```

```
        }
```

```
        sc.close();
```

```
        pw.close();
```

```
}
```

TOPIC NAME: IT 24058

DAY: _____

TIME: _____

DATE: / /

problem 2: difference between static and final,

code:

```
class test {
```

```
    static int x = 10;
```

```
    final int y = 20;
```

```
    static void show () {
```

```
        System.out.println (static method);
```

```
    }
```

```
public class main {
```

```
    public static void main (String[] args) {
```

```
        Test t = new test ();
```

```
        Test.show ();
```

```
        System.out.println (Test.x);
```

```
    }
```

```
}
```

problem 3: factorion number problem.

$$\sum (\text{factorial of digits}) = \text{number}$$

code:

```
import java.util.*;

public class factorion {

    static int factorial(int n) {
        int fact = 1;
        for (int i = 1; i <= n; i++)
            fact = i * fact;
        return fact;
    }

    static boolean isFactorion(int num) {
        int temp = num, sum = 0;

        while (temp > 0) {
            int digit = temp % 10;
            sum = + factorial(digit);
            temp /= 10;
        }
        return sum == num;
    }
}
```


TOPIC NAME: IT 29058

DAY: _____

TIME: _____ DATE: / /

```
public static void main(String[] args) {  
    Scanner sc = new Scanner(System.in);  
    System.out.print("Enter lower bound");  
    int low = sc.nextInt();  
    System.out.print("Enter upper bound");  
    int high = sc.nextInt();  
    System.out.println("Factorion numbers");  
    for (int i = low; i <= high; i++) {  
        if (isFactorion(i))  
            System.out.print(i);  
    }  
}
```

output:

1 2 245 140585

problem 4:- class, instance & local variables
this keyword;

code:-

```
class Demo {
```

```
    static int a = 10;
```

```
    int b = 20;
```

```
    void show (int b) {
```

```
        this.b = b;
```

```
        System.out.println (this.b);
```

```
    }
```

* this refers to current object.

TOPIC NAME: IT 29058

DAY: _____

TIME: _____

DATE: / /

problem 5: method to find sum of Array Element.

code:

```
public class Array sum {  
    static int sumArray (int [] arr) {  
        int sum = 0;  
        for (int x : arr)  
            sum += x;  
        return sum;  
    }  
}
```

```
public static void main (String [] args) {
```

```
    int [] nums = {10, 20, 30};
```

```
    System.out.println ("sum = " + sumArray (nums));  
}
```

output

sum = 60

problem 6:- Access modifiers

Definition:- Access modifiers control visibility of classes, variable, methods.

modifier	same class	same package, subclass	anywhere
public	yes	yes	yes
protected	yes	yes	yes
private	yes	no	no.

problem 7:- smallest positive root of quadratic equation.

for mulla

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

code:

```
import java.util.*;

public class Quadratic {
    public static void main (String[], args) {
```


TOPIC NAME: IF 29058

DAY: _____

TIME: _____

DATE: / /

```
Scanner sc = new Scanner(System.in);
```

```
System.out.print("Enter a b c");
```

```
int a = sc.nextInt();
```

```
int b = sc.nextInt();
```

```
int c = sc.nextInt();
```

```
double d = b*b - 4*a*c;
```

```
if (d < 0) {
```

```
    System.out.println("No real root");
```

```
} else {
```

```
    double r1 = (-b + Math.sqrt(d)) / (2*a);
```

```
    double r2 = (-b - Math.sqrt(d)) / (2*a);
```

```
    double main = Math.min(r1, r2);
```

```
    if (main > 0)
```

```
        System.out.println("Positive root: " + main);
```

```
    }  
}
```

output.

2 5 6 X.

problem 8: Letter, Digit, whitespace. Detection
passing to array to function.

code:

```
public class char check {  
    static void check(char c) {  
        if (char.isLetter(c))  
            System.out.println("Letter");  
        else if (char.isDigit(c))  
            System.out.println("Digit");  
        else if (char.isWhitespace(c))  
            System.out.println("whitespace");  
    }  
    static void printArray(int[] arr) {  
        for (int n: arr)  
            System.out.print(n + " ");  
    }  
    public static void main (String[], args)  
    {  
        check('A');  
    }  
}
```

TOPIC NAME: IT 24058

DAY: _____

TIME: _____

DATE: / /

```
int [] a = {1, 2, 3}
```

```
print array (a);
```

```
}
```

```
{
```

output.

digit.

problem 9: method overriding and super keyword.

Code:

```
class parent {
```

```
    void show() {
```

```
        System.out.println("parent method");
```

```
    }
```

```
}
```

```
class child extends parent {
```

```
    void show() {
```

```
        super.show();
```

```
        System.out.println("child method");
```

```
    }
```

```
}
```

* constructor can not be overridden.

TOPIC NAME: IT 24058

DAY: _____

TIME: _____

DATE: / /

Problem 20: static vs non-static and
palindrome problem.

palindrome program (number and string) code:

```
public class palindrome {  
    static boolean ispalindrome (string s) {  
        string rev = new string builder (s). reverse().  
            to string ();  
        return s. equals (rev);  
    }  
    static boolean ispalindrome (int n) {  
        int rev = 0, temp = n;  
        while (temp > 0) {  
            rev = rev * 10 + temp % 10;  
            temp /= 10;  
        }  
        return rev == n;  
    }  
}
```

TOPIC NAME: IT24058

DAY: _____

TIME: _____

DATE: / /

```
public static void main (String[], args) {  
    System.out.println (is palindrome) ("madam");  
    System.out.println (is palindrome) (121));  
}
```

output
madam
121

TOPIC NAME: IT 24058

DAY: _____

TIME: _____

DATE: / /

problem 11: what is called class abstraction and encapsulation? describe with the example. what are the difference between Abstract class and interface.

solution:-

Abstraction means hiding implementation details and showing only essential features to the user.

Example:

```
abstract class shape {
```

```
    abstract void draw();
```

```
}
```

```
class circle extends shape {
```

```
    void draw() {
```

```
        system.out.println("drawing circle");
```

```
    }
```

PIC NAME: IT24058

DAY: _____

TIME: _____

DATE: / /

encapsulation means wrapping data and methods together into a single unit (class) and protecting data using access modifiers.

Example:

- Difference between Abstract class and interface.

<u>Feature</u>	<u>Abstract</u>	<u>Interface</u>
method	Abstract + concrete	Abstract (default)
variable	instance	public static final only
inheritance	extends	implements.

problem 12: using Inheritance (numerical operation)

1. Base class

```
class Baseclass {  
    void printResult (string msg) {  
        system.out.println(msg);  
    }  
}
```

2. sum class

series:- $1 + 0.9 + 0.8 + \dots + 0.1$

code:-

```
class sumclass extend Baseclass {  
    void calculate sum () {  
        double = 0.0;  
        for (double i = 1.0; i >= 0.1; i -= 0.1) {  
            sum = i + i;  
        }  
        print Result ("sum of series =" + sum);  
    }  
}
```


Q3. GCD and Lcm

code:-

```
class Divisonmultiple extends Base class {
```

```
int gcd (inta, int b) {
```

```
while (b!=0) {
```

```
int r = a % b;
```

```
    a = b;
```

```
    b = r;
```

```
}
```

```
return a;
```

```
}
```

```
void calculate Gcd (int a, int b) {
```

```
int g = gcd (a, b);
```

```
int Lcm = (a * b) / g;
```

```
print result ("GCD" + g);
```

```
print result ("Lcm" + Lcm);
```

```
}
```

output 1

GCD = 12

Lcm = 72

TOPIC NAME: IT 24058

DAY: _____

TIME: _____

DATE: / /

Problem 15:- compare and contrast abstraction achieved through abstract classes and interface.

comparison of Abstract class vs interface.

Feature	Abstract class	interface
Purpose	Partial Abstraction	Full Abstraction.
Method	Abstract + concrete	Abstract (default)
Constructor	allowed	not allowed.
Inheritance	extends (single)	implements (multiple)
State	can maintain state	can not maintain state.

multiple interfaces of example Java code:-

→ class A implements I1, I2 { }

conflicting method signatures.

* If two interfaces have same method signature no problem.

* If two interface have same default method.

problem 16 :

polymorphism and dynamic method Dispatch.

polymorphism means one interface, many implementations. It allows a superclass reference to refer to a subclass object.

Dynamic method Dispatch:- method call is resolved at runtime. Based on the object type, not reference type.

example.

```
class Animal {  
    void sound () {  
        system.out.println (Animal.sound);  
    }  
}  
  
class Dog extends Animal {  
    void sound () {  
        system.out.println (bark);  
    }  
}
```


TOPIC NAME: IT 29058

DAY:

TIME:

DATE: / /

performance impact

* slight runtime overhead (method lookup)

* Trade-off: flexibility vs. maintainability vs
tiny performance cost.

TOPIC NAME: IT 24058

DAY: _____

TIME: _____

DATE: / /

problem 17:-

Arraylist vs linklist.

operation	Array list	linked list.
Access (get)	$O(1)$	$O(n)$
injection (end)	$O(1)$	$O(1)$
Injection (mid)	$O(n)$	$O(1)^*$
Deletion	$O(n)$	$O(1)^*$
memory	less	more.
dataset	large	small

Arraylist:- fast access, large datasets, ready-heavy

linklist:- frequent insert/delete operation

TOPIC NAME: IT 24058

DAY: _____

TIME: _____

DATE: / /

problem 18: customRandom Generator
(Java program).

code:-

```
import java.util.*;
```

```
public class customRandomGenerator {
```

```
    static int [] base = {3, 7, 11, 19, 23};
```

```
    public static void myRand (int a) {
```

```
        long time = System.currentTimeMillis();
```

```
        for (int i=0; i<n; i++) {
```

```
            int r = (int) ((time + base[i] * base.length) % 100);
```

```
            System.out.print (r + " ");
```

```
            time += r;
```

```
        }
```

```
        System.out.println ();
```

```
    }
```


TOPIC NAME: IT 24058

DAY: _____

TIME: _____

DATE: / /

```
public static void myRand () {
```

```
    myRand (5);
```

```
}
```

```
public static void myRand (int n, int mod) {
```

```
    long time = System.nanoTime();
```

```
    for (int i=0; i<n; i++) {
```

```
        int r = (int) ((time + base[i] * base.length) % mod);
```

```
        System.out.print(r + " ");
```

```
        time += r;
```

```
    } System.out.println();
```

```
}
```

```
public static void main (String[], args) {
```

```
    customRandom.myRand (5);
```

```
    customRandom.myRand ();
```

```
    customRandom.myRand (5, 50);
```

```
}
```

TOPIC NAME: IT 24058

DAY: _____

TIME: _____

DATE: / /

problem 19: multithreading in Java.

Thread vs Runnable

Thread	Runnable
Extends Thread class	Implements Runnable.
cannot extend another class	can extend another class
less flexible	more flexible.

Thread safety issues

- Race condition
- Dead lock.
- Starvation.

Ensures only one thread accesses a critical section at a time.

Synchronized void increment() { count++; }

TOPIC NAME: IT 24058

TIME:

DATE:

Deadlock example:

code:-

Thread 1 locks A $\rightarrow$ waits for B

Thread 2 locks B $\rightarrow$ wait for A.

Avoid deadlock.

1. LOCK resources in same order

2. Avoid nested locks.

3. use try lock - C).

TOPIC NAME: IT 24058

DAY: _____

TIME: _____

DATE: / /

problem 20: ~~Expe~~ Exception Handling.

checked vs unchecked exceptions

checked	unchecked
compile-time	runtime
IOException	Null pointer exception
must be handled	optional handling

Custom Exception.

code:-

```
class myException extends Exception {  
    myException (String msg) {  
        super(msg);  
    }  
}
```

TOPIC NAME: IT 24058

DAY: _____

TIME: _____

DATE: / /

Throw vs throws.

throw - actually throws exception.

throw:- declares exception.

code:-

1. void test () throws my exception {

2. throw new my exception (error);

Avoid resource leak.

1. use finally

2. use try-with resources.

code:-

```
try (FileReader fr = new FileReader("a.txt"))  
{
```

problem 21: Default methods in interface (Java 8+) vs Abstract classes.

Effect of default methods

Before Java 8, interface had only abstract method. Since Java 8, interface can have default method with implementation, making them closer to abstract classes.

Can an abstract class implement an interface.

Yes

code:-

```
interface A {
```

```
    default void show () {
```

```
        System.out.println("Interface A");
```

```
    }
```

```
Abstract class B implements A {
```

```
    public void show () {
```

```
        System.out.println("Abstract class");
```

```
    }
```


TOPIC NAME: IT 240 58

DAY: _____

TIME: _____

DATE: / /

conflicting implementations

1. If both interface and Abstract class define the same method.

- class implementation wins.

2. If two interface provide same default method,

- Implementing class must override

Inheritance Rule:-

Class > Abstract class > Interface.

TOPIC NAME: IT 240 58

DAY: _____

TIME: _____

DATE: / /

problem 22% Hashmap vs tree map vs
Linkedhashmap.

Feature	HashMap	LinkedHashMap	Treemap
data structure	Hash Table	Hash table + linked list	Red-Black tree
ordering	no order	Insertion order	sorted order
Time complexity	$O(1)$ avg	$O(1)$ Avg	$O(\log n)$
null keys	one allowed	one allowed	not allowed

When used,

1. Hash map:- fast access, no ordering needed.
2. linkedhashmap:- maintain insertion order.
3. Treemap:- sorted data required.

problem 23: static binding vs dynamic binding (polymorphism).

Relation to polymorphism.

1. Dynamic binding enables runtime polymorphism.
2. static binding does not support polymorphism.

Example:-

```
class parent {  
    static void show () {  
        system.out.println (parent static);  
    }  
    void display () {  
        system.out.println (parent display);  
    }  
}  
class child extends parent {  
    static void show () {  
        system.out.println (child static);  
    }  
}
```



```
public class Test {
```

```
    public static void main (String args) {
```

```
        Parent p = new Child ();
```

```
        p.show ();
```

```
        p.display ();
```

```
    }  
}
```

performance impact

1. static binding → faster (compile-time)

2. dynamic binding → slight overhead (runtime lookup)

3. Trade-off: flexibility vs performance.

TOPIC NAME: IT 24058

DAY: _____

TIME: _____

DATE: / /

problem 24: Executor service vs manual
Thread management.

Advantage of executor service:-

1. Thread reuse (thread pool)
2. Better resource management
3. Easier scalability
4. cleaner code

Callable vs Runnable.

1. Runnable → no return, no ~~exp~~ exception.
2. callable → returns values, throws exception.

Example:

Code:-

```
import java.util.concurrent.*;
```

```
public class Executor demo {
```

```
    public static void main (String [], args)
```

```
{
```

P.T.O

TOPIC NAME : 24044

DAY :

TIME :

DATE :

fix order service with procedure now fixed

fixed pool (2);

callable & memory for () 7.10.12/0;

value & memory pool, 202, submit (1024);

system, out, print(n, pool, got());

ee, shutdown (2);

}

problem 25: Exception Handling Issues + circle program.

Issues to consider

1. proper exception type.
2. meaningful messages.
3. Resource cleanup.
4. Avoid swallowing exceptions.

Circle program code:

```
class circle {
```

```
    private double radius;
```

```
    void setRadius (double r) throws Exception {
```

```
        if (r < 0)
```

```
            throw new Exception (radius cannot be negative);
```

```
            radius = r;
```

```
    }
```

```
    double area () {
```

```
        return math.PI * radius * radius;
```

```
    }
```

```
public class Test {  
    public static void main (String[] args) {  
        Circle c = new Circle();  
        try {  
            c.setRadius (-5);  
        }  
        catch (Exception e) {  
            System.out.println (e.getMessage());  
        }  
    }  
}
```

problem 26: way to create a thread in Java.

Two main ways.

program 1:- using Runnable Interface.

```
class my implements Runnable {
```

```
    public void run() {
```

```
        System.out.println("Thread using Runnable");
```

```
    }
```

```
    public static void main(String[] args) {
```

```
        Thread t = new Thread(new my task());
```

```
        t.start();
```

```
    }
```

```
}
```


TOPIC NAME: IT 240 58

DAY: _____

TIME: _____

DATE: ____ / ____ / ____

Program 28:- Extending Thread class.

code:-

```
class myThread extends Thread {  
    public void run() {  
        System.out.println("Thread using  
        Thread class");  
    }  
    public static void main(String[], args) {  
        myThread t = new myThread();  
        t.start();  
    }  
}
```

TOPIC NAME: IT24058

DAY: _____

TIME: _____

DATE: / /

problem 29:- Read highest value from a file and write the sum of another file (using scanner and print writer)

code:-

```
import java.io.*;  
import java.util.*;
```

```
public class filemaxsum {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(new File("input.txt"));
```

```
        int max = Integer.MIN_VALUE;
```

```
        int sum = 0;
```

```
        while (sc.hasNextInt()) {
```

```
            int n = sc.nextInt();
```

```
            sum += n;
```

```
            if (n > max) max = n;
```

```
        }  
        sc.close();
```

TOPIC NAME: IT 240 58

DAY: _____

TIME: _____

DATE: / /

```
printwriter pw = new printwriter (output.txt)
```

```
pw.println("Higest vale: " + max);
```

```
pw.println("sum: " + sum);
```

```
pw.close();
```

```
{
```


problem 30: Array division with ceiling (n>20)

Source code:-
import java.util.*;

public class - ArrayDivision {

public static void main(String[] args) {

Scanner sc = new Scanner(System.in);

int n = sc.nextInt();

int[] a = new int[n];

int[] b = new int[n/2];

for (int i = 0; i < n; i++) a[i] = sc.nextInt();
for (int i = 0; i < n/2; i++) b[i] = sc.nextInt();

for (int i = 0; i < n; i++) {

int div = (int) Math.ceil(a[i]/b[i]);

```
int rem = a[i] % b[i];
```

```
system.out.println("divisor" + div + "
```

```
remainder" + rem);
```

```
}
```

```
}
```

```
System.out.println("divisor" + div + "
```

```
remainder" + rem);
```

```
System.out.println("divisor" + div + "
```

```
remainder" + rem);
```

```
System.out.println("divisor" + div + "
```

```
remainder" + rem);
```

```
System.out.println("divisor" + div + "
```

problem 31: print current date and Time.

Code:-

```
import java.time.*;  
import java.time.format.*;  
  
public class date-time print {  
    public static void main(String[] args) {  
        Date time formatter f =  
            Date time formatter.of pattern ("dd-mm-yy  
            HH:mm:ss");  
        System.out.println (Local date time: now().  
            format(f));  
    }  
}
```

output

10-01-2026:5:40:-23

TOPIC NAME: Java TIME: 10:00 DATE: 10/10/2020

Problem 32: counter class with static variable

code:-

```
public class counter class {  
    - static int instance count = 0;  
  
    counter () {  
        instance count++;  
  
        if (instance count > 50)  
            instance count = 0;  
    }  
}
```

problem 33: variable argument function.

Source code:-

```
public class Extreme {
```

```
    static int findExtreme (String type, int... nums)
```

```
    {  
        int result = nums[0];
```

```
        for (int n : nums) {
```

```
            if (type.equals ("smallest") && n < result)
```

```
                result = n;
```

```
            if (type.equals ("largest") && n > result)
```

```
                result = n;
```

```
        }
```

```
        return result;
```

```
    }
```

```
    public static void main (String[] args) {
```

TOPIC NAME: IT 24058

DAY: _____

TIME: _____

DATE: / /

```
int x = findExtreme("smallest") {5, 2, 9, 1};  
int y = findExtreme("largest") {8, 3, 10, 4};  
  
system.out.println(x);  
system.out.println(y);  
}
```

output

1	2	5	9
10	8	4	3 1 0

TOPIC NAME: IT24058

DAY: _____

TIME: _____

DATE: / /

problem 34: output the given string code.

- true $\rightarrow$ equals() checks content
- false $\rightarrow$ s1 and s2 are different object
- true $\rightarrow$ s1 and s3 refers to same.

problem 35:

If these methods are not called.

so, no output will be printed

TOPIC NAME: IT 290 54

DAY: _____

TIME: _____

DATE: / /

problem 36: Interface + Abstract Design.

Goal: 5 method.

1. Interface Alpha $\rightarrow$ 2 methods
2. Interface Beta $\rightarrow$ 2 methods
3. Abstract Base implements Alpha and declares 1 abstract method.
4. final class extends Abstract Base.

Code:-

```
interface Alpha {
```

```
    void method 1 ();
```

```
    void method 2 ();
```

```
}
```

```
interface {
```

```
interface Beta {
```

```
    void method 1 ();
```

```
    void method 2 ();
```

```
}
```

DAY: _____

TIME: _____

DATE: / /

TOPIC NAME: IT24058

Abstract Base implement Alpha

abstract class

public void method A1 () {

system.out.println("method A1");

}

public void method A2 () {

system.out.println("method A2");

}

problem 37: Issue with the code.

problem:-

both interface x and y have the same default method show(). Java get, confused.

solution 1:- override the method
code:-

```
public class z implement x, y {
```

```
    public void show() {
```

```
        system.out.println("z's show");
```

```
    }
```

solution 2:- call a specific interface
code:-

```
public class z implement x, y {
```

```
    public void show() {
```

```
        x.sapper.show();
```

```
    }
```

Problem 38:- singleton class program.

code:-

```
class singletonExample {  
    private static singletonExample instance;  
  
    private singletonExample () {  
    }  
  
    public static singletonExample getInstance () {  
        if (instance == null) {  
            instance = new singletonExample ();  
        }  
        return instance;  
    }  
}  
  
public class math {  
    public static void main (String [] args) {  
        singletonExample sa = singletonExample.getInstance();  
    }  
}
```

```

system.out.println(s1 == s2)
}
}

```

Problem 39: Wallet class with custom Exceptions.

Code:- Exceptions.

```

class InvalidAmountEx extends Exception {
    public InvalidAmountEx (String msg) {
        super (msg);
    }
}

```

```

class InsufficientFundsEx extends Exception {
    public InsufficientFundsEx (String msg) {
        super (msg);
    }
}

```


TOPIC NAME : IT24058

DAY: _____

TIME: _____

DATE: / /

wallet class :

```
class wallet {
```

```
    double balance = 0;
```

```
    void addFunds (double amount) throws {
```

```
        if (amount < 0)
```

```
            balance += amount;
```

```
    }
```

```
    void spend (double amount)
```

```
    {
        if (amount < 0)
```

```
            throw new InvalidAmountException ("no");
```

```
        if (balance - amount < 0)
```

```
            throw new InsufficientException ("no");
```

```
            balance -= amount;
```

```
    }
```

```
}
```

Test program:

Code:-

```
public class Testwallet {
```

```
    public static void main (String [] args) {
```

```
        Wallet w = new wallet ();
```

```
        try {
```

```
            w.addFunds (-50);
```

```
        }
```

```
        catch (Exception e) {
```

```
            System.out.println (e.getMessage());
```

```
        }
```

```
    try {
```

```
        w.addFunds (200);
```

```
        w.spend (200);
```

```
    }
```

```
    catch (Exception e) {
```

```
        System.out.println (e.getMessage());
```

```
    }
```

```
}
```

TOPIC NAME: IT24058

DAY: _____

TIME: _____

DATE: / /

problem 90: command-line calculator
program.

code:-

```
public class calculator {
```

```
    public static void main(String[] args){
```

```
        try {
```

```
            if (args.length < 2)
```

```
                throw new Exception("Enter two num");
```

```
            int a = Integer.parseInt(args[0]);
```

```
            int b = Integer.parseInt(args[1]);
```

```
            System.out.println("sum:" + (a+b));
```

```
            System.out.println("diff:" + (a-b));
```

```
            System.out.println("product:" + (a*b));
```

```
            System.out.println("Quotient" + (a/b));
```



```

    }
    catch (Exception e) {

```

system.out.println("Invalid input

```

    }
}

```

output:-

10 5

sum = 15

diff = 5

product = 50

quotient = 2.